

符号学习赋能自演化智能体

Wangchunshu Zhou^{*}, Yixin Ou^{*}, Shengwei Ding^{*}, Long Li, Jialong Wu, Tiannan Wang,

Jiamin Chen, Shuai Wang, Xiaohua Xu, Ningyu Zhang, Huajun Chen, Yuchen Eleanor Jiang[†]

AIWaves Inc.

{chunshu,eleanor}@aiwaves.cn  <https://github.com/aiwaves-cn/agents>

摘要

人工智能社区一直在通过开发“语言智能体”来探索通向通用人工智能（AGI）的路径。语言智能体是同时涉及提示技术与工具使用方法的复杂大语言模型（LLM）流水线。尽管语言智能体已在许多现实任务上展现出令人印象深刻的能力，当前语言智能体研究仍存在一个根本局限：它以模型为中心，或者说以工程为中心。也就是说，语言智能体在提示、工具和流水线方面的进步，需要人类专家投入大量手工工程工作，而不是从数据中自动学习。我们认为，从以模型或工程为中心转向以数据为中心，也就是让语言智能体具备在环境中自主学习和演化的能力，是它们有可能实现 AGI 的关键。

本文提出智能体符号学习：一个使用符号优化器、让语言智能体以数据为中心自主优化自身的系统框架。具体而言，我们把智能体视为符号网络，其中可学习的权重由提示、工具以及它们的堆叠方式定义。智能体符号学习通过模拟联结主义学习中的两种基础算法——反向传播与梯度下降——来优化语言智能体内部的符号网络。与数值权重不同，智能体符号学习处理的是权重、损失和梯度在自然语言中的对应物。我们在标准基准和复杂现实任务上开展概念验证实验，结果表明，智能体符号学习使语言智能体能够在创建并部署到真实环境之后继续更新自身，从而形成“自演化智能体”。我们展示了智能体符号学习框架的潜力，并将整个框架开源，以促进未来对以数据为中心的智能体学习研究。

1 引言

大语言模型的近期进展 [Radford et al., 2018, 2019, Brown et al., 2020, Ouyang et al., 2022, OpenAI, 2023, Touvron et al., 2023a,b]，为构建能够自主解决复杂任务的语言智能体带来了可能。开发 AI 智能体的常见做法，是将复杂任务分解为由提示和工具堆叠而成的 LLM 流水线 [Park et al., 2023, Hong et al., 2023, Zhou et al., 2023b, Chen et al., 2023b, Xie et al., 2023]。从某种意义上说，语言智能体可以被视为连接联结主义 AI（即智能体的 LLM 骨干）与符号主义 AI（即提示和工具组成的流水线）的 AI 系统；这也部分解释了它们在现实问题求解场景中的有效性。

然而，当前语言智能体的发展受制于这样一个事实：针对特定任务构建并定制语言智能体系统需要大量工程投入。具体而言，研究人员和开发者必须手工把复杂任务分解为对 LLM 更容易处理的子任务——本文称之为节点——然后再为特定节点精心设计提示与工具，包括 API 函数、知识库、记忆等。这一过程的复杂性，使当前语言智能体研究呈现出以模型为中心或以工程为中心的格局。这意味着，研究人员几乎无法像以数据为中心训练神经网络那样，在数据集上手工调节或优化语言智能体。它既限制了手工编码语言智能体的鲁棒性和通用性，也使智能体适配新任务或新数据分布时需要付出大量工程成本。我们认为，从以工程为中心的语言智能体开发转向以数据为中心的学习，是语言智能体研究的重要一步。

为此，近期已有一些工作尝试自动优化语言智能体。例如，DSPy [Khattab et al., 2023] 提出通过自举或在不同提示组件的组合空间中随机搜索，算法化地优化 LLM 提示；GPTSwarm [Zhuge et al., 2024] 进一步提出以迭代优化过程处理 DSPy 所带来的组合优化挑战。Agent-pro [Zhang et al., 2024b] 提出在竞争环境中优化与智能体内部策略相对应的提示组件。AgentOptimizer [Zhang et al., 2024a] 则提出用精心设计的提示来优化函数。尽管这

^{*} 共同第一作者。

[†] 通讯作者。

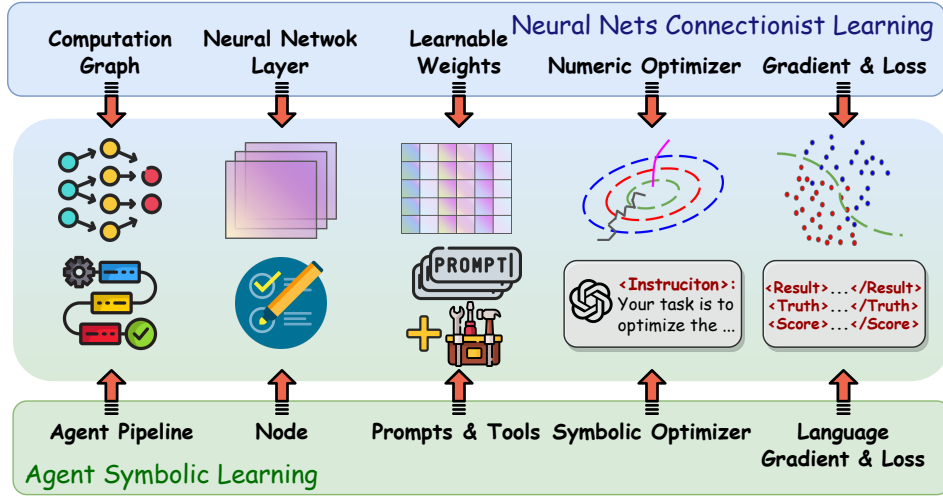


图 1: 智能体符号学习与神经网络联结主义学习之间的类比。

些方法在某些场景中有效，它们通常只优化智能体系统中的独立模块，例如某个特定节点的提示。因此，这些优化方法容易陷入孤立提示、工具或节点各自的局部最优，进而损害整个智能体系统的性能。这类似于早期训练神经网络时逐层独立优化的做法 [Hinton and Salakhutdinov, 2006]；如今看来，整体优化神经网络能够获得更好的性能几乎是不言自明的。我们认为，智能体优化同样如此：联合优化智能体内全部符号组件，是优化智能体的关键。

本文提出一个用于训练语言智能体的智能体符号学习框架。该框架受神经网络训练所采用的联结主义学习过程 [Hinton, 1990] 启发。具体来说，我们在语言智能体与神经网络之间建立如下类比：智能体的流水线对应神经网络的计算图；流水线中的节点对应神经网络中的层；节点所用的提示和工具则对应一层的权重。由此，我们可以在智能体训练语境下，用基于语言的损失、梯度和权重来实现联结主义学习的主要组成部分，即反向传播和基于梯度的权重更新。具体而言，我们使用精心设计的提示流水线，实现智能体训练中的损失函数、反向传播和权重优化器。对于一个训练样本，框架首先执行“前向传播”（即运行智能体），并将每个节点的输入、输出、提示和工具使用情况存入一条“轨迹”。随后，我们用基于提示的损失函数评估结果，得到“语言损失”。之后，沿轨迹从最后一个节点向第一个节点反向传播语言损失，为每个节点中的符号组件生成文本分析与反思；我们称之为语言梯度。最后，再通过另一套精心设计的提示，依据语言梯度更新各节点中的全部符号组件，以及由节点及其连接构成的计算图。该方法也天然支持优化多智能体系统：既可以把不同智能体视为不同节点，也可以允许多个智能体在同一节点内采取行动。

智能体符号学习框架是一种模拟标准联结主义学习过程的智能体学习框架。不同于分别优化单个提示或工具的既有方法，智能体符号学习联合优化智能体系统中的全部符号组件，包括提示、工具，以及把它们堆叠成智能体系统的流水线。这种自顶向下的优化方案还使智能体符号学习能够“整体性”地优化智能体系统，避免各独立组件的局部最优。这使面向复杂现实问题的语言智能体能够有效地从数据中学习，从而为语言智能体研究由以工程为中心转向以数据为中心打开了可能。此外，由于基于语言的损失函数在生成语言损失时不要求真实标签，我们的框架使语言智能体能够从经验中学习，并在创建、部署到真实环境之后有意识地更新自身的所有符号组件，从而实现“自演化智能体”¹。

作为概念验证，我们在标准 LLM 基准和复杂智能体任务上开展了一系列实验。结果表明，所提出的智能体符号学习框架能够有效优化、设计提示与工具，并通过从训练数据中学习来更新整体智能体流水线。我们开源了智能体符号学习框架的全部代码和提示，以推动未来对以数据为中心的智能体学习研究。

¹ 智能体也可以在真实环境中收集训练数据，并通过微调更新 LLM 骨干。这样，智能体中的全部组件都可以更新。我们将其留作未来工作。

2 相关工作

2.1 语言模型、提示与语言智能体

语言模型是一类经过训练、用于评估词或词元序列概率的机器学习模型。大语言模型 (LLM) [Radford et al., 2018, 2019, Brown et al., 2020, Ouyang et al., 2022, OpenAI, 2023, Touvron et al., 2023a,b] 通常是指采用自回归概率分解方案、由 Transformer 架构 [Vaswani et al., 2017] 参数化、拥有大量参数并在大规模语料上训练的语言模型。随着模型规模、训练数据和计算量的扩展, LLM 在生成类人文本和理解上下文方面展现出卓越能力。

另一方面, 提示是释放 LLM 能力的关键。提示是控制 LLM 行为与输出的关键组件, 也是人与 LLM 之间的接口。提示设计会显著影响语言模型性能; 提示工程已经取得许多进展, 包括上下文学习 [Brown et al., 2020]、思维链提示 [Nye et al., 2022, Wei et al., 2022]、ReAct [Yao et al., 2022]、自我精炼 [Madaan et al., 2023]、自洽性 [Wang et al., 2023]、循环提示 [Zhou et al., 2023a] 等。

语言智能体进一步扩展了语言模型的功能: 它允许 LLM 使用工具 [Schick et al., 2023], 并将 LLM 集成到能够执行多步任务的更广泛系统中 [Park et al., 2023, Hong et al., 2023, Zhou et al., 2023b, Chen et al., 2023b, Xie et al., 2023]。通过把提示与工具堆叠为精心设计的流水线, 智能体可以广泛应用于从客服自动化到高级数据分析的不同场景。

2.2 从自动提示工程到智能体优化

随着提示工程在学术界和工业界日益普及, 近期许多工作研究了如何自动化提示工程过程。例如, Pryzant et al. [2020] 和 Yang et al. [2024] 使用精心设计的提示, 释放 LLM 自行开展提示工程的能力。另一方面, Prasad et al. [2023] 和 Guo et al. [2024] 则采用遗传算法等不同搜索算法来优化提示。

由于提示是智能体的关键组件, 自动提示工程的成功为自动智能体优化打开了可能。与自动提示工程类似, 智能体优化方法也可以分为两类: 基于提示的方法和基于搜索的方法。例如, Agent-pro [Zhang et al., 2024b] 和 AgentOptimizer [Zhang et al., 2024a] 借助精心设计的提示, 优化智能体流水线某一节点中的提示或工具。这些方法作用于智能体内的孤立组件。另一条研究路线探索了基于搜索的智能体优化算法。Sordoni et al. [2023] 使用变分推断优化堆叠的 LLM; DSPy [Khattab et al., 2023] 用搜索算法在组合空间中寻找最佳提示或节点; GPTSwarm [Zhuge et al., 2024] 进一步改进了这一组合优化问题的搜索算法。这些方法存在若干主要局限。第一, 只有当指标能够通过可编码的方程进行数值定义时, 搜索算法才主要适用; 然而, 大多数智能体任务都是复杂的现实问题, 其成功与否无法用某些方程定义, 例如软件开发或创意写作。第二, 这些方法分别更新各组件, 因此会受每个节点或组件局部最优的影响。它们也不具备在流水线中添加节点或实现新工具的能力。相比之下, 本文提出的智能体符号学习框架是首批能够“整体性”优化智能体系统的智能体学习方法之一, 可以优化提示、工具、节点, 以及它们堆叠成智能体的方式。

此外, 近期也有一些工作通过合成数据来微调智能体的 LLM 骨干 [Chen et al., 2023a, Qiao et al., 2024, Song et al., 2024]。这条研究路线与本文工作正交, 我们认为二者可以互补。ICE [Qian et al., 2024] 也研究了语言智能体的跨任务迁移学习, 可以与本文方法互补, 用于构建自演化智能体。

3 智能体符号学习

3.1 问题形式化

我们首先通过类比神经网络训练中的组件和过程, 对智能体符号学习框架进行形式化。下面定义框架的关键组件, 并说明本节使用的符号。

如图 2 所示, 智能体符号学习框架受神经网络训练所采用的联结主义学习过程 [Hinton, 1990] 启发。我们先借助与联结主义学习框架的类比, 引入关键概念的记号:

- **智能体流水线 $\mathcal{A}$:** 神经网络中的计算图表示各层的结构及其连接; 与之类似, 智能体流水线表示智能体处理输入数据时依次经过的节点 (或步骤)。它由一系列节点 $\{\mathcal{N}_1, \mathcal{N}_2, \dots, \mathcal{N}_n\}$ 构成, 这些节点分阶段处理输入数

Algorithm 1 智能体符号学习框架

Require: $\mathcal{I}$ ▷ 智能体系统的输入
Require: $\mathcal{A}$ ▷ 包含节点的智能体流水线
Require: $\mathcal{G}$ ▷ 基于提示的梯度传播函数
Require: $\mathcal{L}$ ▷ 基于提示的损失函数
Ensure: 智能体系统中更新后的符号组件

```

1:  $\tau \leftarrow []$  ▷ 初始化轨迹
2: 前向传播
3: for 每个  $\mathcal{N} \in \mathcal{A}$  do
4:    $\mathcal{I}_n \leftarrow$  获取  $\mathcal{N}$  的输入 ▷ 节点输入
5:    $\mathcal{O}_n \leftarrow \mathcal{N}(\mathcal{I}_n, \mathcal{P}_n, \mathcal{T}_n)$  ▷ 节点输出
6:   将  $(\mathcal{I}_n, \mathcal{O}_n, \mathcal{P}_n, \mathcal{T}_n)$  追加到  $\tau$ 
7: end for
8: 损失计算
9:  $\mathcal{L}_{\text{lang}} \leftarrow \mathcal{L}(\tau)$  ▷ 计算语言损失
10: 反向传播
11: for 每个  $\mathcal{N} \in \text{reverse}(\mathcal{A})$  do
12:    $\nabla_{\text{lang}}^n \leftarrow \mathcal{G}(\nabla_{\text{lang}}^{n+1}, \mathcal{I}_n, \mathcal{O}_n, \mathcal{P}_n, \mathcal{T}_n, \mathcal{L}_{\text{lang}})$  ▷ 对最后一个节点,  $\nabla_{\text{lang}}^{n+1} = \emptyset$ 
13:   将  $\nabla_{\text{lang}}^n$  追加到  $\tau$ 
14: end for
15: 权重更新
16: for 每个  $\mathcal{N} \in \mathcal{A}$  do
17:   使用  $\nabla_{\text{lang}}^n$  更新  $\mathcal{P}_n, \mathcal{T}_n$  ▷ 更新提示和工具
18: end for
19: 使用  $\{\nabla_{\text{lang}}^n\}$  更新  $\mathcal{A}$  ▷ 更新智能体流水线
20: return  $(\mathcal{A}, \mathcal{P}, \mathcal{T})$  ▷ 更新后的智能体系统

```

据。需要注意，在某些智能体框架中，节点会在执行期间动态分配，因此智能体流水线依赖于输入；这与动态神经网络的情形相似。

- **节点 $\mathcal{N}$** : 智能体流水线中的一个独立步骤。节点在智能体中的作用，类似于神经网络中的一层。节点 $\mathcal{N}_n$ 接收同样采用自然语言形式的**节点输入** $\mathcal{I}_n$ 。一般而言，一个节点的输入由前一节点的输出以及（可选的）环境输入（例如人类输入）组成。节点 $\mathcal{N}_n$ 使用 LLM，并结合**提示** $\mathcal{P}_n$ 和**工具** $\mathcal{T}_n$ ²处理输入 $\mathcal{I}_n$ 。输出 $\mathcal{O}_n$ 采用自然语言形式，并传递给下一个节点。
- **轨迹 τ** : 轨迹的作用类似于神经网络的计算图。它存储前向传播期间的全部信息，包括每个节点的输入、输出、提示和工具使用情况，并负责梯度的反向传播。
- **语言损失 $\mathcal{L}_{\text{lang}}$** : 智能体符号学习框架中的语言损失与神经网络损失相似，因为二者都衡量期望结果与实际结果之间的差异。主要区别在于：语言损失采用文本形式，由精心设计的提示所实现的自然语言损失函数生成；传统损失则是由数值方程形式的损失函数计算出的浮点数。
- **语言梯度 ∇_{lang}** : 与梯度在联结主义学习中的作用类似，语言梯度是相对于语言损失、用于更新智能体中各组件的文本分析与反思。

3.2 智能体符号学习过程

定义关键组件后，我们可以用算法 1 概括智能体符号学习框架的工作流程。本节将详细说明其中的每一步。

² $\mathcal{T}_n$ 包括工具使用时的输入与输出，以及工具本身的实现。

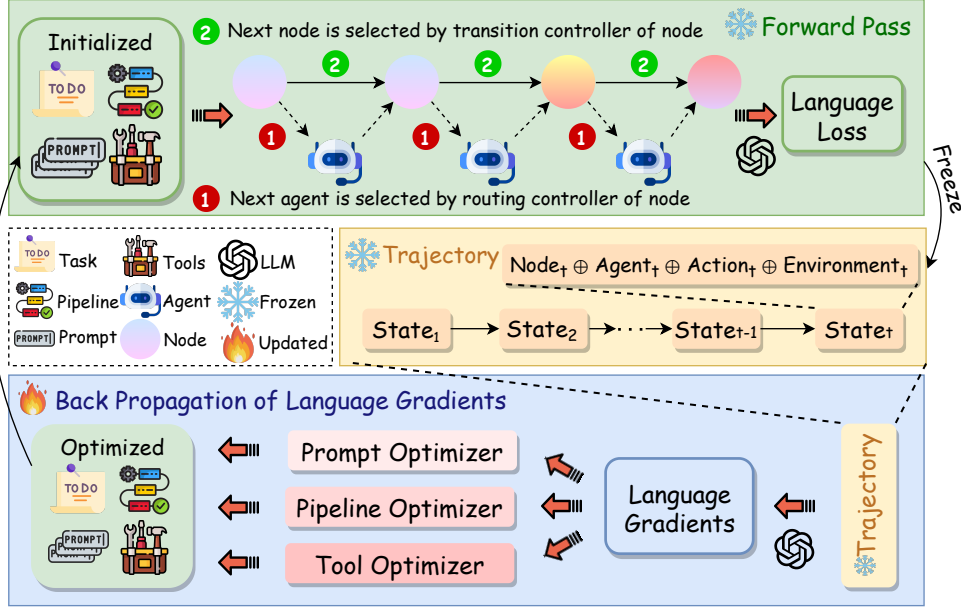


图 2: 智能体符号学习框架示意图。

前向传播 前向传播与标准的智能体执行几乎完全相同。主要区别是，我们会把输入、提示、工具使用情况和输出保存到轨迹中，供语言梯度反向传播使用。这类似于 PyTorch [Paszke et al., 2019] 和 TensorFlow [Abadi et al., 2016] 等深度学习框架：它们会把中间输出和激活值存储在神经网络的计算图中。

语言损失计算 完成前向传播后，我们把轨迹输入 LLM，并使用精心设计的提示模板 $\mathcal{P}_{\text{loss}}$ ，为一个训练样本计算语言损失：

$$\mathcal{L}_{\text{lang}} = \text{LLM}(\mathcal{P}_{\text{loss}}(\tau)) \quad (1)$$

关键在于提示模板的设计：它应当相对于输入、环境和任务要求，对智能体表现进行整体性评估。为此，我们精心设计了用于语言损失计算的提示模板，它由以下部分组成：任务描述、输入、轨迹、少样本示例、原则和输出格式控制。其中，任务描述、输入和轨迹依赖于数据，而少样本示例、原则和输出格式控制对所有任务及训练样本保持固定。语言损失同时包含自然语言评论和数值分数（后者同样通过提示生成）。生成语言损失时，我们可以选择把输入对应的真实标签一并提供；我们把这种情形称为有监督智能体学习。框架也可以不依赖真实标签，而是根据任务描述评估输出和轨迹并生成语言损失；此时可称智能体正在进行无监督智能体学习，这使语言智能体能够自我演化。该提示模板的具体实现见附录中的原文材料。

语言梯度的反向传播 在标准联结主义学习中，梯度反向传播旨在计算各权重对总体损失的影响，以便优化器据此更新权重。类似地，我们也为语言梯度设计了一种“反向传播”算法。具体来说，我们从最后一个节点反向迭代至第一个节点，并用精心设计的提示让 LLM 为每个节点计算梯度：

$$\nabla_{\text{lang}}^n = \text{LLM}(\mathcal{P}_{\text{gradient}}(\nabla_{\text{lang}}^{n+1}, \mathcal{I}_n, \mathcal{O}_n, \mathcal{P}_n, \mathcal{T}_n, \mathcal{L}_{\text{lang}})) \quad (2)$$

提示模板 $\mathcal{P}_{\text{gradient}}$ 用于指示 LLM 生成语言梯度，即针对节点内符号组件的分析与反思。受反向传播思想启发，我们把当前节点之后所执行节点的语言梯度，以及轨迹中保存的当前节点执行信息，一并提供给 LLM。也就是说，在分析和反思时，LLM 不仅要考虑提示和工具是否适合当前节点的子目标，还必须考虑它们如何影响下一节点子目标的完成。通过从后向前逐一链接，所有节点的语言梯度都与智能体的整体成功相关，并对其负责。该方法有效降低了分别优化各孤立提示与工具时趋向局部最优的风险，从而改善智能体系统的整体性能。

基于语言梯度的更新 框架的最后一步，是借助语言梯度更新各节点中的提示和工具，并优化整体智能体流水线。这通过“符号优化器”完成。符号优化器是精心设计的提示流水线，能够优化智能体的符号权重。我们构建了三类符号优化器：PromptOptimizer、ToolOptimizer 和 PipelineOptimizer。相关提示的具体实现见附录中的原文材料。

PromptOptimizer: 为便于提示优化，我们把提示拆分为不同组件，包括任务描述、少样本示例、原则和输出格式控制。随后，针对每一种提示组件的优化分别设计提示。所有这些提示都包含详细说明和示范，告诉 LLM 在推理如何编辑原始提示组件时，应当怎样聚焦语言梯度。

ToolOptimizer: ToolOptimizer 是一条提示流水线。它首先指示 LLM 决定应采用哪类操作：是改进工具（通过编辑函数调用所用的工具描述）、删除工具，还是实现新工具。随后，ToolOptimizer 会调用分别为工具编辑、删除和创建而设计不同提示。

PipelineOptimizer: PipelineOptimizer 的目标，是优化由节点及其连接构成的智能体流水线。提示首先介绍用于定义智能体流水线的智能体编程语言（本文采用 Zhou et al. [2023b] 所提出的智能体编程语言）。接着，提示描述若干可供 LLM 更新流水线的原子操作定义，包括添加、删除和移动节点。然后，它指示 LLM 先分析流水线可以怎样改进，再用这些原子操作实现更新。智能体编程语言以及更新流水线所用原子操作的详细说明，见附录中的原文材料。

由于上述优化器都在自然语言空间中运行，而部分优化操作需要在代码空间中完成，我们采用一种简单策略：任何非法更新最多重试三次；若错误仍然存在，就丢弃该更新。我们还采用回滚策略：优化后重新运行当前样本；如果基于语言的损失函数所评估的性能下降，则回滚到原始智能体。此外，我们在优化器的每个提示中都加入一个“学习率”组件，用来控制 LLM 优化提示、工具和智能体流水线时的激进程度。

批量训练 上述优化方案每次处理一个训练样本，类似于随机梯度下降。受小批量随机梯度下降在实践中效果更好、或更稳定这一事实启发，我们也为符号优化器设计了批量训练变体。具体来说，先分别对每个样本执行前向传播、损失计算和反向传播；随后，把同一节点的一批语言梯度共同输入 LLM，并通过提示让 LLM 在更新智能体时整体考虑所有这些语言梯度。

4 实验

4.1 设置

4.1.1 任务

我们既在标准 LLM 基准上开展实验，也在更复杂的智能体任务上进行实验。任务、数据集和评估指标如下。

表 1: 标准 LLM 基准上的结果。 HotPotQA 单元格依次报告精确匹配/F1；MATH 报告准确率；HumanEval 报告 Pass@1。

方法	HotPotQA		MATH		HumanEval	
	GPT-3.5	GPT-4	GPT-3.5	GPT-4	GPT-3.5	GPT-4
GPTs	24 / 38.8	33 / 44.3	23.2	53.1	59.2	71.7
Agents	27 / 37.5	39 / 49.8	23.8	56.0	59.5	85.0
Agents w/ AutoPE	29 / 39.8	38 / 50.3	22.5	57.2	63.5	82.3
DSPy	35 / 43.9	40 / 50.5	17.3	48.4	66.7	77.3
本文方法	35 / 44.8	41 / 54.0	38.8	60.7	64.5	85.8

标准基准 我们在 HotpotQA [Yang et al., 2018]、MATH [Hendrycks et al., 2021] 和 HumanEval [Chen et al., 2021] 等标准 LLM 基准上开展实验。HotPotQA 是一项多跳问答任务，对丰富的背景知识提出挑战。由于我们发现数据集的“hard”划分对语言智能体更具挑战性，因此使用该划分。MATH 收录了具有挑战性的数学竞赛题。HumanEval 是一个评估集，要求 LLM 或智能体根据文档字符串合成程序。评估指标方面，HotPotQA 使用 F1 和精确匹配，MATH 使用准确率，HumanEval 使用 Pass@1。我们在这些数据集上禁用工具，以保证结果能够与既有文献进行有意义的比较。

复杂智能体任务 我们把**创意写作**和**软件开发**视为两项复杂智能体任务。对于创意写作任务，我们遵循Yao et al. [2023] 的设置，向智能体提供 4 个随机句子，要求它们写出一篇连贯的 4 段文章，并让 4 个段落分别以这 4 个输入句子结尾。该任务具有开放性和探索性，既考验创造性思维，也考验高层规划。我们遵循Yao et al. [2023]，使用 GPT-4 评分来评估文章。另一方面，软件开发任务要求智能体系统根据一份简单的产品需求文档（PRD）开发出可执行软件。我们依据所生成软件的可执行性评估各对比智能体；该指标用 1 到 4 的数值分数衡量，对应逐级提高的执行能力。具体来说，1 分表示执行失败，2 分表示代码能够成功运行，3 分表示符合预期工作流程，4 分表示与预期完全一致。

4.1.2 基线

我们将所提出的方法与以下基线进行比较：

- **GPTs**：一种简单基线，使用 GPT 和精心设计的提示；
- **Agents**：使用 Agents [Zhou et al., 2023b] 框架³实现的语言智能体方法，配有精心设计的提示、工具和流水线；
- **DSPy**：一种 LLM 流水线优化框架，能够搜索提示组件的最佳组合。对于评估指标无法用方程和代码定义的复杂智能体任务，它并不适用；
- **Agents + AutoPE**：一个变体，其中智能体流水线各节点的提示由 LLM 按照Yang et al. [2024] 所述方法进行优化。该变体不涉及语言梯度反向传播，也不涉及基于语言梯度的优化。

我们同时报告 GPT-3.5 和 GPT-4 的结果。GPT-3.5 使用 gpt-3.5-turbo-0125 端点，GPT-4 使用 gpt-4-turbo-0409 端点。对于本文方法，我们以 **Agents** 基线为起点，再在其上开展智能体符号学习。

4.2 结果

表 2：软件开发任务结果。

任务	GPTs	Agents	本文方法
Flappy Bird	2	2	3
坦克大战	1	2	4
2048	1	2	4
贪吃蛇	2	3	4
打砖块	2	3	4
平均分	1.6	2.4	3.8

表 3：创意写作任务结果。

方法	GPT-3.5	GPT-4
GPTs	4.0	6.0
Agents	4.2	6.0
Agents w/ AutoPE	4.4	6.5
ToT	3.8	6.8
本文方法	6.9	7.4

LLM 基准上的结果 标准 LLM 基准上的结果见表 1。可以看到，所提出的智能体符号学习框架相对于所有对比方法均取得了一致提升；其中，在竞赛级基准 MATH 上的性能增益尤其显著。相比之下，传统的基于 LLM 的提示优化方法（Agents w/ AutoPE）与基于搜索的提示优化方法（DSPy）并不稳定：它们在某些情形下带来良好提升，在另一些情形下却导致性能显著下降。这表明，智能体符号学习框架更加鲁棒，能够更有效地优化语言智能体的整体性能。

复杂任务上的结果 软件开发和创意写作的结果分别见表 2和表 3。可以看到，本文方法在两项任务上均显著优于所有对比基线，而且性能差距甚至大于传统 LLM 基准上的差距。有趣的是，在创意写作任务上，本文方法甚至优于思维树（tree-of-thought）；后者是一种经过精心设计的提示工程与推理算法。我们发现，本文方法成功找到了规划、写作和修订流水线，并且各步骤的提示都得到了良好优化。我们还发现，智能体符号学习框架恢复出与 MetaGPT [Hong et al., 2023] 中专门面向软件开发而设计的标准操作流程相似的流程。这验证了所提出智能体符号学习框架在现实任务上的有效性：这些任务没有真实标签，整体性能也无法用方程或代码计算，因而不同于 DSPy 等基于搜索的算法所适用的情形。

³我们也测试了 OpenAgents 和 AgentVerse 等其他智能体框架，并得到相似结果。

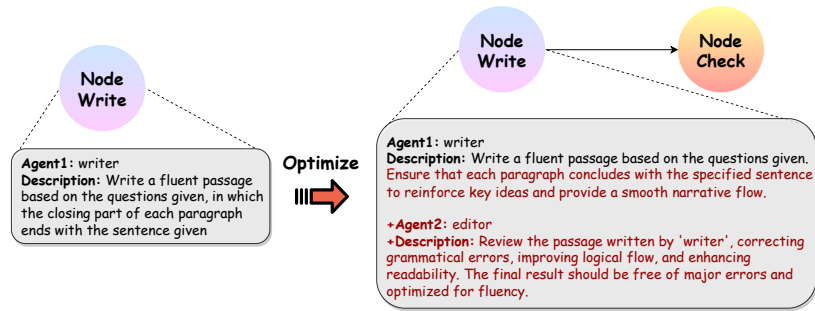


图 3: 在创意写作任务上开展的一项案例研究。

4.3 案例研究与分析

接下来，图 3 给出了智能体符号学习框架优化动态的案例研究。可以看到，本文方法能够像人类专家开发语言智能体那样，有效开展提示工程并设计智能体流水线。

此外，我们发现，与神经网络初始化对训练十分重要类似，智能体系统的初始化也会对最终性能产生不可忽视的影响。总体上，我们发现，以最简单的方式初始化智能体、再让符号优化器执行优化通常是有帮助的。相反，如果初始智能体系统经过度工程化，性能往往会变得不稳定。由这一观察自然引出的扩展方向是：或许可以先在大规模、多样化任务上开展某种预训练，为通用智能体得到一种用途广泛的初始化，再用智能体符号学习将其适配到专门任务。我们还发现，与由准确率或 F1 评估性能的标准基准相比，本文方法在复杂现实任务上的成功更加显著且稳定。这表明，未来的智能体学习研究应当更多聚焦现实任务；智能体研究社区也应致力于构建侧重智能体学习评估、由多样化复杂智能体任务组成的基准，并研究稳健衡量进展的方法。

5 结论

本文提出智能体符号学习，一个联合优化智能体系统内全部符号组件的智能体学习框架。智能体符号学习框架从标准联结主义学习过程汲取灵感，以开展符号学习。它使用基于语言的损失、梯度和优化器，相对于智能体系统的整体性能优化提示、工具与智能体流水线。所提出框架是优化能够借助复杂流水线解决现实复杂任务的智能体的早期尝试之一。我们的框架让语言智能体能够“从数据中学习”，并在创建、部署到真实环境之后“自我演化”。我们开展了若干概念验证实验，结果表明，智能体符号学习框架能够在不同任务复杂度下有效优化智能体。我们认为，智能体研究从以模型为中心转向以数据为中心，是迈向通用人工智能具有意义的一步；为加速这一转变，我们开源了智能体符号学习框架的代码和提示。

参考文献

- Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, et al. {TensorFlow}: a system for {Large-Scale} machine learning. In *12th USENIX symposium on operating systems design and implementation (OSDI 16)*, pages 265–283, 2016.
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901. Curran Associates, Inc., 2020. URL <https://proceedings.neurips.cc/paper/2020/file/1457c0d6bfc4967418bfb8ac142f64a-Paper.pdf>.
- Baian Chen, Chang Shu, Ehsan Shareghi, Nigel Collier, Karthik Narasimhan, and Shunyu Yao. Fireact: Toward language agent fine-tuning, 2023a.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pondé de Oliveira Pinto, Jared Kaplan, Harrison Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf,

- Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Joshua Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. *CoRR*, abs/2107.03374, 2021. URL <https://arxiv.org/abs/2107.03374>.
- Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chen Qian, Chi-Min Chan, Yujia Qin, Yaxi Lu, Ruobing Xie, et al. Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors in agents. *arXiv preprint arXiv:2308.10848*, 2023b.
- Qingyan Guo, Rui Wang, Junliang Guo, Bei Li, Kaitao Song, Xu Tan, Guoqing Liu, Jiang Bian, and Yujiu Yang. Connecting large language models with evolutionary algorithms yields powerful prompt optimizers. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=ZG3RaNI08>.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *NeurIPS*, 2021.
- Geoffrey E Hinton. Connectionist learning procedures. In *Machine learning*, pages 555–610. Elsevier, 1990.
- Geoffrey E Hinton and Ruslan R Salakhutdinov. Reducing the dimensionality of data with neural networks. *science*, 313 (5786):504–507, 2006.
- Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiwu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. Metagpt: Meta programming for a multi-agent collaborative framework, 2023.
- Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. Dspy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*, 2023.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Sean Welleck, Bodhisattwa Prasad Majumder, Shashank Gupta, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback, 2023.
- Maxwell Nye, Anders Johan Andreassen, Guy Gur-Ari, Henryk Michalewski, Jacob Austin, David Bieber, David Dohan, Aitor Lewkowycz, Maarten Bosma, David Luan, Charles Sutton, and Augustus Odena. Show your work: Scratchpads for intermediate computation with language models, 2022. URL <https://openreview.net/forum?id=iedYJm92o0a>.
- OpenAI. GPT-4 technical report, 2023.
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Gray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In Alice H. Oh, Alekh Agarwal, Danielle Belgrave, and Kyunghyun Cho, editors, *Advances in Neural Information Processing Systems*, 2022. URL <https://openreview.net/forum?id=TG8KACxEON>.
- Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior, 2023.
- Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.
- Archiki Prasad, Peter Hase, Xiang Zhou, and Mohit Bansal. GRIPS: Gradient-free, edit-based instruction search for prompting large language models. In Andreas Vlachos and Isabelle Augenstein, editors, *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*, pages 3845–3864, Dubrovnik, Croatia, May 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.eacl-main.277. URL <https://aclanthology.org/2023.eacl-main.277>.
- Reid Pryzant, Richard Diehl Martinez, Nathan Dass, Sadao Kurohashi, Dan Jurafsky, and Diyi Yang. Automatically neutralizing subjective bias in text. In *The Thirty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2020, The Thirty-Second Innovative Applications of Artificial Intelligence Conference, IAAI 2020, The Tenth AAAI Symposium on*

- Educational Advances in Artificial Intelligence, EAAI 2020, New York, NY, USA, February 7-12, 2020*, pages 480–489. AAAI Press, 2020. doi: 10.1609/AAAI.V34I01.5385. URL <https://doi.org/10.1609/aaai.v34i01.5385>.
- Cheng Qian, Shihao Liang, Yujia Qin, Yining Ye, Xin Cong, Yankai Lin, Yesai Wu, Zhiyuan Liu, and Maosong Sun. Investigate-consolidate-exploit: A general strategy for inter-task agent self-evolution, 2024.
- Shuofei Qiao, Ningyu Zhang, Runnan Fang, Yujie Luo, Wangchunshu Zhou, Yuchen Eleanor Jiang, Chengfei Lv, and Huajun Chen. AUTOACT: automatic agent learning from scratch via self-planning. *CoRR*, abs/2401.05268, 2024. doi: 10.48550/ARXIV.2401.05268. URL <https://doi.org/10.48550/arXiv.2401.05268>.
- Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. Improving language understanding by generative pre-training. 2018.
- Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. URL <https://openreview.net/forum?id=Yacmpz84TH>.
- Yifan Song, Da Yin, Xiang Yue, Jie Huang, Sujian Li, and Bill Yuchen Lin. Trial and error: Exploration-based trajectory optimization for LLM agents. *CoRR*, abs/2403.02502, 2024. doi: 10.48550/ARXIV.2403.02502. URL <https://doi.org/10.48550/arXiv.2403.02502>.
- Alessandro Sordoni, Xingdi Yuan, Marc-Alexandre Côté, Matheus Pereira, Adam Trischler, Ziang Xiao, Arian Hosseini, Friederike Niedtner, and Nicolas Le Roux. Joint prompt optimization of stacked LLMs using variational inference. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. URL <https://openreview.net/forum?id=iImnbUVhok>.
- Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models, 2023a.
- Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. Llama 2: Open foundation and fine-tuned chat models, 2023b.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=1PL1NIMMrw>.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35: 24824–24837, 2022.
- Tianbao Xie, Fan Zhou, Zhoujun Cheng, Peng Shi, Luoxuan Weng, Yitao Liu, Toh Jing Hua, Junning Zhao, Qian Liu, Che Liu, Leo Z. Liu, Yiheng Xu, Hongjin Su, Dongchan Shin, Caiming Xiong, and Tao Yu. Openagents: An open platform for language agents in the wild, 2023.
- Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=Bb4VGOWELI>.
- Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In Ellen Riloff, David Chiang, Julia

- Hockenmaier, and Jun'ichi Tsujii, editors, *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380, Brussels, Belgium, October–November 2018. Association for Computational Linguistics. doi: 10.18653/v1/D18-1259. URL <https://aclanthology.org/D18-1259>.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models, 2023.
- Shaokun Zhang, Jieyu Zhang, Jiale Liu, Linxin Song, Chi Wang, Ranjay Krishna, and Qingyun Wu. Offline training of language model agents with functions as learnable weights, 2024a.
- Wenqi Zhang, Ke Tang, Hai Wu, Mengna Wang, Yongliang Shen, Guiyang Hou, Zeqi Tan, Peng Li, Yueting Zhuang, and Weiming Lu. Agent-pro: Learning to evolve via policy-level reflection and optimization, 2024b.
- Wangchunshu Zhou, Yuchen Eleanor Jiang, Peng Cui, Tiannan Wang, Zhenxin Xiao, Yifan Hou, Ryan Cotterell, and Mrinmaya Sachan. Recurrentgpt: Interactive generation of (arbitrarily) long text, 2023a.
- Wangchunshu Zhou, Yuchen Eleanor Jiang, Long Li, Jialong Wu, Tiannan Wang, Shi Qiu, Jintian Zhang, Jing Chen, Ruipu Wu, Shuai Wang, Shiding Zhu, Jiyu Chen, Wentao Zhang, Ningyu Zhang, Huajun Chen, Peng Cui, and Mrinmaya Sachan. Agents: An open-source framework for autonomous language agents, 2023b.
- Mingchen Zhuge, Wenyi Wang, Louis Kirsch, Francesco Faccio, Dmitrii Khizbullin, and Jurgen Schmidhuber. Language agents as optimizable graphs. *arXiv preprint arXiv:2402.16823*, 2024.